

Security Audit

Haiku (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	10
Intended Smart Contract Functions	11
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Haiku team partnered with Hashlock to conduct a security audit of their Haiku Architecture and Haiku Intents Smart Contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

The core purpose of Haiku Intents is to act as a seamless bridge between the well-established trading paradigms of TradFi and the evolving DeFi ecosystem - simplifying the process of portfolio construction and management. Haiku Intents does this by abstracting away the numerous financial primitives and DeFi protocols and allowing the user to focus purely on their trading outcomes. This intents platform can also be leveraged to let users move into, out of, or between any of our Haiku vaults seamlessly as part of their trading. Haiku aims to create a comprehensive and user-friendly DeFi ecosystem.

Project Name: Haiku Architecture and Haiku Intents Smart Contracts

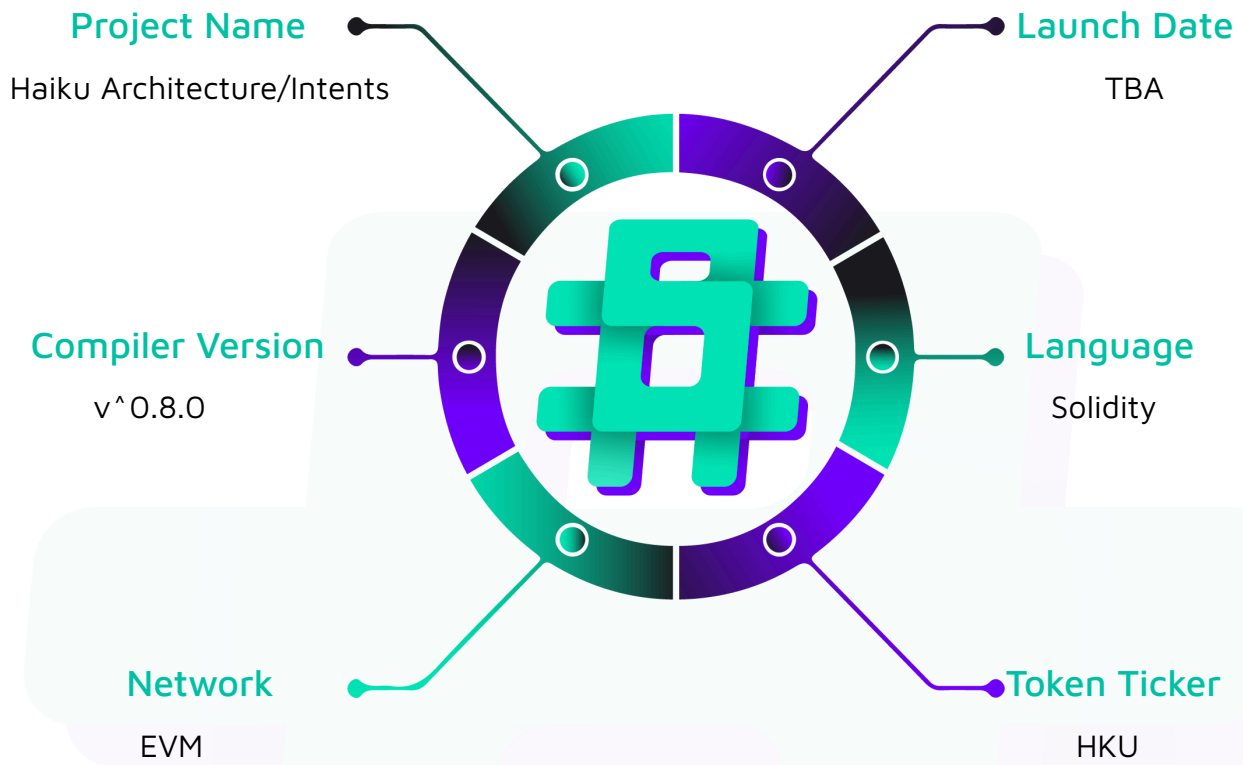
Project Type: Defi

Compiler Version: ^0.8.0

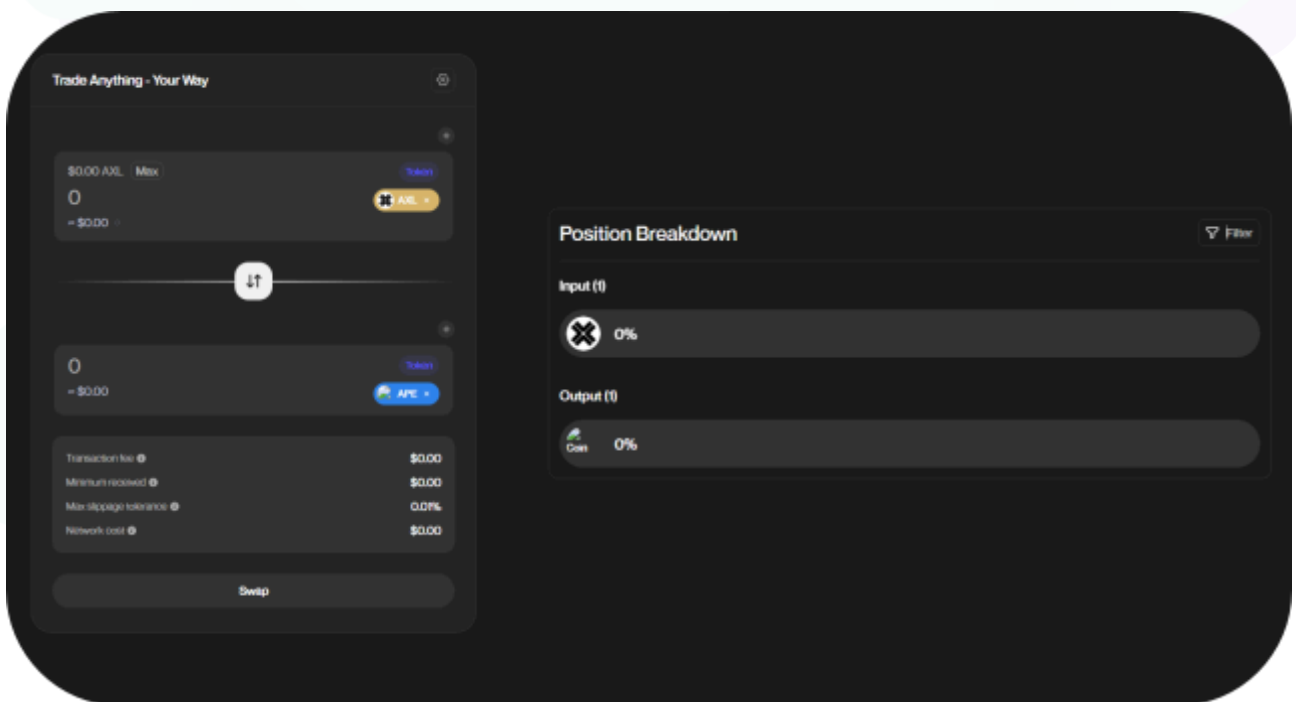
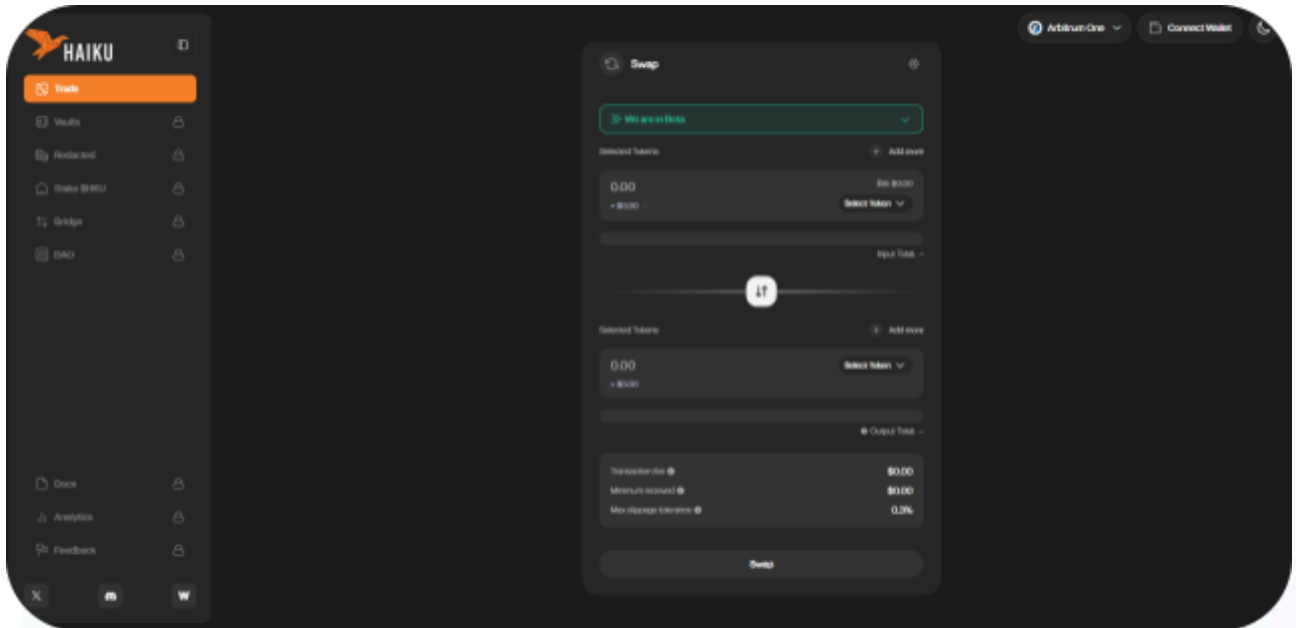
Website: <https://www.haiku.trade/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within both the Haiku Architecture and the Haiku Intents contracts, and the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Haiku Protocol Smart Contracts
Platform	EVM / Solidity
Audit Date	February, 2025
Contract 1	Agent.sol
Contract 1 MD5 Hash	3784593cadd9a24cf7a01c250917a048
Contract 2	AgentImplementation.sol
Contract 2 MD5 Hash	f9a03014c73b6c46d98e090fcad2de83
Contract 3	Router.sol
Contract 3 MD5 Hash	fff23cb45f624bb4b1dd026ffceb5271
Contract 4	AaveV2FlashLoanCallback.sol
Contract 4 MD5 Hash	42c2427a32dfc0ae0403d53eb89cdca3
Contract 5	AaveV3FlashLoanCallback.sol
Contract 5 MD5 Hash	590f11ed2e3db60e8b09a16f8558dc56
Contract 6	BalancerV2FlashLoanCallback.sol
Contract 6 MD5 Hash	7048b2a9a0d758cf80669abe71c353a5
Contract 7	CallbackFeeBase.sol
Contract 7 MD5 Hash	fa6f4bb6801ce4d178d8720e31a03547
Contract 8	MorphoFlashLoanCallback.sol
Contract 8 MD5 Hash	85ebaa51c1e5ba64e232f409ca35539a
Contract 9	RadiantV2FlashLoanCallback.sol

Contract 9 MD5 Hash	3eeabf9046276f2c192fef88970e7d71
Contract 10	SparkFlashLoanCallback.sol
Contract 10 MD5 Hash	f92eff90635652f53107e1c789097a44
Contract 11	BoringVault.sol
Contract 11 MD5 Hash	25748845e308bce373d3b271a42a1280
Contract 12	BaseDecoderAndSanitizer.sol
Contract 12 MD5 Hash	e7d0611018dc5fdd7ccacc55e0b656e8
Contract 13	ProtocolinkDecoderAndSanitizer.sol
Contract 13 MD5 Hash	45641aa619f5673fcb1cdedcfebf9d4f
Contract 14	BoringDrone.sol
Contract 14 MD5 Hash	e6ec688c037bdd0c2f789a20102b4a73
Contract 15	DroneLib.sol
Contract 15 MD5 Hash	1c6de30a786304390808c737bfb88d64
Contract 16	BoringGovernance.sol
Contract 16 MD5 Hash	1077e21cc78e94d572e38960332d2954
Contract 17	AccountantWithRateProviders.sol
Contract 17 MD5 Hash	a4fcdb79c6fa8798acfa2077d96a71f1
Contract 18	ManagerWithMerkleVerification.sol
Contract 18 MD5 Hash	cbeed9f5ce67f47c79a70810a293306b
Contract 19	Pauser.sol
Contract 19 MD5 Hash	3a0da67c89b5dc4b592ad3059d441a6d
Contract 20	TellerWithMultiAssetSupport.sol
Contract 20 MD5 Hash	ab674dfea0de028e81ec422ef09efb95
Contract 21	WithdrawQueue.sol
Contract 21 MD5 Hash	bced5f8967fcc02ddd089c3c9d63f381
Contract 22	AddressToBytes32Lib.sol

Contract 22 MD5 Hash	06c1fd7f8a76cea29ae50c411975e8c1
Contract 23	Deployer.sol
Contract 23 MD5 Hash	2dc40fb38ad9564a557ad267e237a02c
Contract 24	GenericRateProvider.sol
Contract 24 MD5 Hash	88d04bd40bed73cc854c3f13851e7743
Contract 25	IncentiveDistributor.sol
Contract 25 MD5 Hash	ff086de5207c65c6efcd4fb75dc5c526
Contract 26	PaymentSplitter.sol
Contract 26 MD5 Hash	418615366ced227c15ca88b7f7eaae87
Contract 27	Signature.sol
Contract 27 MD5 Hash	ae2a93891a1c82b0f53abbfe774fe876
Contract 28	DepositLib.sol
Contract 28 MD5 Hash	a659ac2560b8d6dd00b6536de9b93f5d
Contract 29	WithdrawLib.sol
Contract 29 MD5 Hash	99ab36532b1fa75b65d3549f8eea8771
GitHub Commit Hash (Intents)	e744a69e3e8b19abae2570ec7f3c9802355bab91
GitHub Commit Hash (Architecture)	9fa022b4a19225b9119df251dcf1ce27bca45ac3

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow both simple and intricate logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin and Solmate contracts. We initially identified some vulnerabilities that have since been addressed.

**Not Secure****Vulnerable****Secure****Hashlocked**

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Low severity vulnerabilities

1 Gas Optimization

3 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Vault Contracts (BoringVault fork) - Core purpose: Handles asset management and strategy execution - <code>BoringVault.sol</code>: The main vault contract that holds user assets and manages shares - <code>AccountantWithRateProviders.sol</code>: Manages exchange rates and fees - <code>TellerWithMultiAssetSupport.sol</code>: Handles deposits and withdrawals with support for multiple assets - <code>ManagerWithMerkleVerification.sol</code>: Validates and executes vault management operations using merkle proofs - Various decoders and sanitizers for different protocols (Aave, Balancer, Curve etc.)	Contracts achieve this functionality.
Intents Contracts (Protocolink fork) - Core purpose: Handles user transaction bundling and execution - <code>Router.sol</code>: Main entry point for all intent executions - <code>Agent.sol</code>: Individual user contracts that execute the bundled transactions - <code>AgentImplementation.sol</code>: Core logic for executing bundles of transactions - Various callback contracts for flash loans from different protocols - Helper libraries for fees, approvals, etc.	Contracts achieve this functionality.

Code Quality

This audit scope involves the smart contracts of the Haiku Protocol, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Haiku Protocol smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] PaymentSplitter#constructor - Missing validation for the total percentage

Description

The smart contract lacks a validation mechanism to ensure that the total percentage value across all split percentages is exactly 100%.

Recommendation

Add a validation check in the constructor to ensure that the total percentage value equals 100%.

Status

Resolved

[L-02] BoringGovernance#setBeforeTransferHook, setShareLocker, TellerWithMultiAssetSupport#setShareLockPeriod - Missing events

Description

The above functions are missing events when key states are updated.

Recommendation

Add a relevant event in the function.

Status

Resolved

Gas

[G-01] PaymentSplitter#payoutSplits - Insufficient storage access in loop due to uncached array length

Description

In the `payoutSplits` function, the loop reads the `splits` array length from storage during every iteration.

By not caching the array length into a memory variable, the contract incurs unnecessary gas costs, especially when the loop iterates multiple times.

Recommendation

Cache the array length into a memory variable before entering the loop.

Status

Resolved

QA

[Q-01] Router - Missing maximum delegation expiry limit creates potentially permanent delegations

Description

The Router contract's `allow` and `allowBySig` functions permit setting delegation expiry timestamps with no upper bound. Delegations can be set to any value up to `type(uint128).max`, effectively creating permanent delegations that could remain valid for decades unless revoked by the user.

```
function allow(address delegatee, uint128 expiry) public {  
  
    delegations[msg.sender][delegatee].expiry = expiry;  
  
    emit Delegated(msg.sender, delegatee, expiry);  
}
```

```
function allowBySig(  
    DataType.DelegationDetails calldata details,  
    address delegator,  
    bytes calldata signature  
) external {  
  
    _validateDelegationDetails(details, delegator, signature);  
  
    address delegatee = details.delegatee;  
    uint128 expiry = details.expiry;  
  
    delegations[delegator][delegatee].updateAll(expiry, details.nonce);  
  
    emit Delegated(delegator, delegatee, expiry);  
}
```

Impact

While not a direct vulnerability since users can revoke delegations at any time via `disallow`, the lack of a maximum expiry limit can potentially have several security implications. Users might accidentally set extremely long or effectively permanent delegations, or compromised private keys could give attackers a very long access window. There is no way to mass-revoke all old delegations except individually, and setting `type(uint128).max` effectively creates permanent permissions, while users may not realize the implications of setting far-future expiry timestamps.

Recommendation

Add a maximum delegation duration limit to both `allow` and `allowBySig`, something resembling the following:

```
// Add to Router.sol

/// @notice Maximum delegation expiry duration (e.g. 1 year)
uint128 public constant MAX_DELEGATION_DURATION = 365 days;

function allow(address delegatee, uint128 expiry) public {
    require(expiry <= block.timestamp + MAX_DELEGATION_DURATION, "Expiry too far");
    delegations[msg.sender][delegatee].expiry = expiry;
    emit Delegated(msg.sender, delegatee, expiry);
}

function allowBySig(
    DataType.DelegationDetails calldata details,
    address delegator,
    bytes calldata signature
) external {
```

```

    require(details.expiry <= block.timestamp + MAX_DELEGATION_DURATION, "Expiry too
far");

    _validateDelegationDetails(details, delegator, signature);

    address delegatee = details.delegatee;

    uint128 expiry = details.expiry;

    delegations[delegator][delegatee].updateAll(expiry, details.nonce);

    emit Delegated(delegator, delegatee, expiry);
}

```

Status

Resolved

[Q-02] WithdrawQueue - Missing minimum validation for non-zero completion window

Description

In `WithdrawQueue`, while there is a `DEFAULT_COMPLETION_WINDOW` of 7 days that acts as a fallback, the functions that set the completion window lack validation to prevent setting unreasonably low non-zero values. Even though such parameters are set by trusted roles, all major DeFi protocols are shifting toward rigorous input validation as a good practice to prevent operational errors.

Current implementation:

```

function setupWithdrawAsset(
    ERC20 asset,

    uint32 withdrawDelay,

    uint32 completionWindow,

    uint16 withdrawFee,

    uint16 maxLoss

) external requiresAuth {

```

```

WithdrawAsset storage withdrawAsset = withdrawAssets[asset];

if (withdrawFee > MAX_WITHDRAW_FEE) revert WithdrawQueue__WithdrawFeeTooHigh();

if (maxLoss > MAX_LOSS) revert WithdrawQueue__MaxLossTooLarge();

if (withdrawAsset.allowWithdraws) revert WithdrawQueue__AlreadySetup();

withdrawAsset.allowWithdraws = true;

withdrawAsset.withdrawDelay = withdrawDelay;

withdrawAsset.completionWindow = completionWindow;

withdrawAsset.withdrawFee = withdrawFee;

withdrawAsset.maxLoss = maxLoss;
}

function changeCompletionWindow(ERC20 asset, uint32 completionWindow) external
requiresAuth {
    WithdrawAsset storage withdrawAsset = withdrawAssets[asset];

    if (!withdrawAsset.allowWithdraws) revert WithdrawQueue__WithdrawsNotAllowed();

    withdrawAsset.completionWindow = completionWindow;
}

```

Recommendation

Add validation to ensure non-zero completion windows are reasonably long, something resembling the following:

```

uint32 private constant MIN_COMPLETION_WINDOW = 1 hours;

if (completionWindow > 0 && completionWindow < MIN_COMPLETION_WINDOW) {
    revert WithdrawQueue__CompletionWindowTooShort();
}

```

Status

Resolved

[Q-03] Contracts - Floating pragma**Description**

Some contracts have `pragma solidity ^0.8.0` and it might allow the contracts to be deployed with a different version than the one used for testing.

Different pragma versions being used in test and mainnet may pose unidentified security issues.

Recommendation

Specify a specific version of Solidity in the pragma statement.

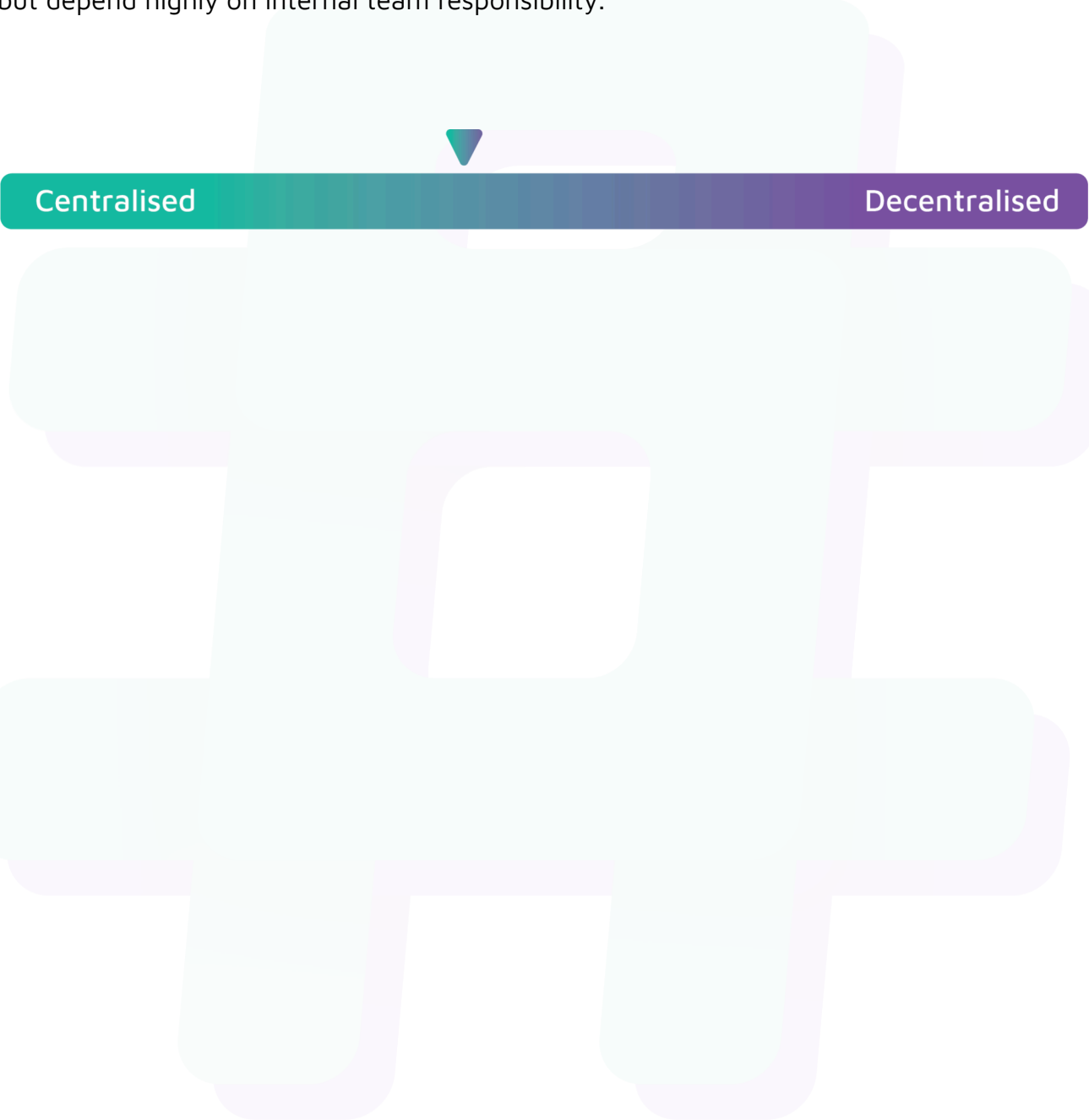
Status

Resolved

Centralisation

The Haiku Protocol values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Haiku's project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.